MINI FULL-STACK MERN PROJECT

Student Management System (CRUD)

Stack:

- Frontend: **React**
- Backend: **Node.js + Express**
- Database: **MongoDB**
- Data flow: **REST API**

---

1. PROJECT OVERVIEW

This is a full-stack MERN application where React is used for UI, Node.js and Express handle backend APIs, and MongoDB stores student data.

---

2. PROJECT STRUCTURE (IMPORTANT)

```
student-mern/
|
 ├── backend/
|   ├── models/
|   |   └── Student.js
|   ├── routes/
|   |   └── studentRoutes.js
|   ├── index.js
|   └── package.json
|
└── frontend/
    ├── src/
   |   ├── App.js
   |   └── index.js
```

```
└── package.json
```

---

3. BACKEND SETUP (Node + Express + MongoDB)

Step 1: Create Backend

```
mkdir backend
cd backend
npm init -y
npm install express mongoose cors
```

---

backend/server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

//app.use(cors());
app.use(cors({
  origin: "http://localhost:3000",
  methods: ["GET", "POST", "PUT", "DELETE"],
  allowedHeaders: ["Content-Type"]
}));
app.use(express.json());

mongoose
  .connect("mongodb://127.0.0.1:27017/collegeDB")
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));
```

```
app.use("/api/students", require("./routes/studentRoutes"));

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

---

backend/models/Student.js

```
const mongoose = require("mongoose");

const studentSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  course: {
    type: String,
    enum: ["CSE", "CSM"],   // ✅ dropdown restriction
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  year: {
    type: Number,
```

```
    required: true
  }
});

module.exports = mongoose.model("Student", studentSchema);
```

---

backend/routes/studentRoutes.js

```
const express = require("express");
const router = express.Router();
const Student = require("../models/Student");

// ➕ CREATE student
router.post("/", async (req, res) => {
  try {
    const student = new Student(req.body);
    await student.save();
    res.json(student);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// 📄 READ all students
router.get("/", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});
```

```
// 🔍 SEARCH student (by name OR email OR course)
router.get("/search", async (req, res) => {
  try {
    const { query } = req.query;

    const students = await Student.find({
      $or: [
        { name: { $regex: query, $options: "i" } },
        { email: { $regex: query, $options: "i" } },
        { course: { $regex: query, $options: "i" } }
      ]
    });

    res.json(students);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
//update Students
router.put("/:id", async (req, res) => {
  try {
    const updatedStudent = await Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );
    res.json(updatedStudent);
```

```
  } catch (error) {
    res.status(500).json({ message: "Update failed" });
  }
});
//Delete Students
router.delete("/:id", async (req, res) => {
  try {
    await Student.findByIdAndDelete(req.params.id);
    res.json({ message: "Student deleted" });
  } catch (error) {
    res.status(500).json({ message: "Delete failed" });
  }
});

module.exports = router;
```

Run Backend

node index.js

Output:

MongoDB Connected

Server running on port 5000

---

4. FRONTEND SETUP (React)

Step 2: Create React App

Open **new terminal**:

npx create-react-app frontend

cd frontend

```
npm install axios
npm start
```

📄 **frontend/src/App.js**

```
import { useEffect, useState } from "react";
import "./App.css";

function App() {
  const [student, setStudent] = useState({
    name: "",
    course: "",
    email: "",
    year: ""
  });

  const [students, setStudents] = useState([]);
  const [search, setSearch] = useState("");
  const [editId, setEditId] = useState(null);

  // FETCH ALL
  const fetchStudents = async () => {
    const res = await fetch("http://localhost:5000/api/students");
    const data = await res.json();
    setStudents(data);
  };

  useEffect(() => {
    fetchStudents();
```

```
}, []);

const handleChange = (e) => {
  setStudent({ ...student, [e.target.name]: e.target.value });
};

// ADD or UPDATE
const handleSubmit = async (e) => {
  e.preventDefault();

  if (editId) {
    // UPDATE
    await fetch(`http://localhost:5000/api/students/${editId}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(student)
    });
  } else {
    // ADD
    await fetch("http://localhost:5000/api/students", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(student)
    });
  }

  setStudent({ name: "", course: "", email: "", year: "" });
  setEditId(null);
```

```
  fetchStudents();
};

// LOAD DATA INTO FORM
const handleEdit = (s) => {
  setStudent({
    name: s.name,
    course: s.course,
    email: s.email,
    year: s.year
  });
  setEditId(s._id);
};

// DELETE
const handleDelete = async (id) => {
  await fetch(`http://localhost:5000/api/students/${id}`, {
    method: "DELETE"
  });
  fetchStudents();
};

// SEARCH
const handleSearch = async () => {
  const res = await fetch(
    `http://localhost:5000/api/students/search?query=${search}`
  );
  const data = await res.json();
```

```
    setStudents(data);
  };

  return (
    <div className="container">
      <h2>🎓 Student MERN CRUD</h2>

      <form onSubmit={handleSubmit}>
        <input name="name" placeholder="Name" value={student.name}
onChange={handleChange} required />

        <select name="course" value={student.course} onChange={handleChange} required>
          <option value="">Select Course</option>
          <option value="CSE">CSE</option>
          <option value="CSM">CSM</option>
        </select>

        <input name="email" placeholder="Email" value={student.email}
onChange={handleChange} required />
        <input name="year" placeholder="Year" value={student.year}
onChange={handleChange} required />

        <button type="submit">
          {editId ? "Update Student" : "Add Student"}
        </button>
      </form>

      <div className="search-box">
```

```
      <input placeholder="Search" value={search} onChange={(e) =>
setSearch(e.target.value)} />
      <button onClick={handleSearch}>Search</button>
      <button onClick={fetchStudents}>Reset</button>
    </div>

    <table>
      <thead>
        <tr>
          <th>Name</th>
          <th>Course</th>
          <th>Email</th>
          <th>Year</th>
          <th>Actions</th>
        </tr>
      </thead>
      <tbody>
        {students.map((s) => (
          <tr key={s._id}>
            <td>{s.name}</td>
            <td>{s.course}</td>
            <td>{s.email}</td>
            <td>{s.year}</td>
            <td className="actions">
              <button className="edit-btn" onClick={() => handleEdit(s)}>✏️ Edit</button>
              <button className="delete-btn" onClick={() => handleDelete(s._id)}>🗑️
Delete</button>
            </td>
```

```
          </tr>
        ))}
      </tbody>
    </table>
  </div>
 );
}

export default App;
```

---

frontend/src/index.js (default, no change)

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

```
body {
  background-color: #f4f6f9;
  font-family: Arial, Helvetica, sans-serif;
}

.container {
  max-width: 900px;
  margin: auto;
```

```
  background-color: #ffffff;
  padding: 20px;
  margin-top: 40px;
  border-radius: 10px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

h2 {
  text-align: center;
  color: #2c3e50;
}

form {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  justify-content: center;
}

input,
select {
  padding: 8px;
  width: 180px;
  border-radius: 5px;
  border: 1px solid #ccc;
}

button {
```

```
  padding: 8px 14px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  background-color: #3498db;
  color: white;
}

button:hover {
  opacity: 0.9;
}

.search-box {
  margin-top: 20px;
  text-align: center;
}

table {
  width: 100%;
  margin-top: 20px;
  border-collapse: collapse;
}

th {
  background-color: #34495e;
  color: white;
  padding: 10px;
}
```

```
td {
  padding: 8px;
  text-align: center;
  border: 1px solid #ddd;
}

tr:nth-child(even) {
  background-color: #ecf0f1;
}
td button {
  margin: 2px;
  font-size: 14px;
}
.actions {
  display: flex;
  gap: 6px;
  justify-content: center;
}

.edit-btn {
  background-color: #f39c12;
  color: white;
}

.delete-btn {
  background-color: #e74c3c;
  color: white;
```

}

Run Frontend

npm start

Browser opens:

http://localhost:3000

---

5. FINAL OUTPUT

UI:

Student Management

[Name] [Course] [E-mail] [Year] [Add]

[Search] [Reset] [Edit] [Delete]

Anil - CSE – anil@gmail.com 2026

Sunita – CSM sunitha@gmail.com 2023

✓ Enter name
✓ Enter course

✓ Enter Email

✓ Enter Year
✓ Click **Add**
✓ Data stored in MongoDB
✓ Data fetched & displayed

---

6. COMPLETE DATA FLOW

React UI

|

Axios (HTTP)

|

Express API

|

Mongoose

|

MongoDB

---

7. DEMONSTRATION(SCRIPT)

1️⃣ Start MongoDB
2️⃣ Run backend → show API
3️⃣ Run frontend → show UI
4️⃣ Add student
5️⃣ Show MongoDB collection
6️⃣ Refresh page → data persists

This is a MERN stack project where React handles the frontend, Node.js and Express provide REST APIs, MongoDB stores data, and Axios is used for communication.